INTRODUCTION TO SQL
PART - 2

Join Queries

A join query joins *some* of the rows in one table with *some* of the rows in another

When constructing a join, it is necessary to think very carefully about

- which tables are to be joined
- which rows are to be joined with which

QTY
200
700
200
200
500
600
400
800
100
200
500
300
300
1000
200
100
1200
800
400
500
100
200
500

For each part supplied, get part number and names of all cities supplying the part.

Select P.Pno, City

From P, SPJ

where SPJ.Pno = P.Pno;

P				
PNO	PNAME	PRICE	COLOR	WEIG
P1	NUT	20.00	RED	
P2	BOLT	14.00	GREEN	
P3	SCREW	14.00	BLUE	
P4	SCREW	11.00	RED	
P5	CAM	150.00	BLUE	
P6	COG	12.00	RED	

Note the use of the . to resolve the ambiguity

Get all (part name, project name) pairs such that the part and the project in question are located in the same city

```
SELECT  PNAME, JNAME
FROM P, J
WHERE P.CITY = J.CITY;
```

J		
JNO	**JNAME**	**CITY**
J1	SORTER	KANDY
J2	PUNCH	MATARA
J3	READER	JAFFNA
J4	CONSOLE	JAFFNA
J5	COLLATOR	COLOMBO
J6	VIDEO	GALLE
J7	TAPE	COLOMBO

P					
PNO	**PNAME**	**PRICE**	**COLOR**	**WEIGHT**	**CITY**
P1	NUT	20.00	RED	12	COLOMBO
P2	BOLT	14.00	GREEN	17	KANDY
P3	SCREW	14.00	BLUE	17	MATARA
P4	SCREW	11.00	RED	14	COLOMBO
P5	CAM	150.00	BLUE	12	KANDY
P6	COG	12.00	RED	19	COLOMBO

PNAME	JNAME
NUT	COLLATOR

Get all info about the (supplier, part) pairs where the supplier and part concerned are located in the same city, but omitting suppliers with status less than 20

```
SELECT S.*,  P.*
FROM S, P
WHERE S.CITY = P.CITY
AND STATUS >= 20;
```

S			
SNO	**SNAME**	**STATUS**	**CITY**
S1	SAMAN	20	COLOMBO
~~S2~~	~~JAGATH~~	~~10~~	~~KANDY~~
S3	ANIL	30	KANDY
S4	GAMINI	20	COLOMBO
S5	BALA	30	JAFFNA

P					
PNO	**PNAME**	**PRICE**	**COLOR**	**WEIGHT**	**CITY**
P1	NUT	20.00	RED	12	COLOMBO
P2	BOLT	14.00	GREEN	17	KANDY
P3	SCREW	14.00	BLUE	17	MATARA
P4	SCREW	11.00	RED	14	COLOMBO
P5	CAM	150.00	BLUE	12	KANDY
P6	COG	12.00	RED	19	COLOMBO

Get all supplier/supplying information of those suppliers with status value less than 20, and the quantity supplied greater than 200

```
SELECT S.*, SPJ.*
FROM S, SPJ
WHERE S.Sno = SPJ.Sno
AND STATUS < 20
AND QTY > 200;
```

S			
SNO	**SNAME**	**STATUS**	**CITY**
S1	SAMAN	20	COLOMBO
S2	JAGATH	10	KANDY
S3	ANIL	30	KANDY
S4	GAMINI	20	COLOMBO
S5	BALA	30	JAFFNA

SPJ			
SNO	**PNO**	**JNO**	**QTY**
S1	P1	J1	200
S1	P2	J4	700
S2	P3	J2	200
S2	P3	J3	200
S2	P3	J4	500
S2	P3	J5	600
S2	P3	J6	400
S2	P3	J7	800
S2	P5	J2	**100**
S3	P3	J3	200
S3	P4	J4	500
S4	P2	J1	300
S4	P2	J4	300
S5	P1	J2	1000
S5	P2	J2	200
S5	P2	J4	100
S5	P3	J4	1200
S5	P4	J4	800
S5	P5	J4	400
S5	P5	J5	500
S5	P5	J7	100
S5	P6	J2	200
S5	P6	J4	500

The value of a shipment is obtained from the price of the part being supplied, and the quantity.

For each shipment, get the supplier number, part number, project number, and the value of the shipment.

```
SELECT SPJ.Sno, SPJ.Pno, SPJ.Jno, SPJ.QTY*P.PRICE AS VALUE
FROM SPJ, P
WHERE SPJ.Pno = P.Pno;
```

P					
PNO	**PNAME**	**PRICE**	**COLOR**	**WEIGHT**	**CITY**
P1	NUT	20.00	RED	12	COLOMBO
P2	BOLT	14.00	GREEN	17	KANDY
P3	SCREW	14.00	BLUE	17	MATARA
P4	SCREW	11.00	RED	14	COLOMBO
P5	CAM	150.00	BLUE	12	KANDY
P6	COG	12.00	RED	19	COLOMBO

SPJ			
SNO	**PNO**	**JNO**	**QTY**
S1	P1	J1	200
S1	P2	J4	700
S2	P3	J2	200
S2	P3	J3	200
S2	P3	J4	500
S2	P3	J5	600
S2	P3	J6	400
S2	P3	J7	800
S2	P5	J2	100

Get all pairs of city names such that a supplier located in the first city is currently supplying a part stored in the second city

```
SELECT DISTINCT S.CITY, P.CITY
FROM S, SPJ, P
WHERE S.Sno = SPJ.Sno
AND SPJ.Pno = P.Pno;
```

SPJ			
SNO	**PNO**	**JNO**	**QTY**
S1	P1	J1	200
S1	P2	J4	700
S2	P3	J2	200
S2	P3	J3	200
S2	P3	J4	500
S2	P3	J5	600
S2	P3	J6	400
S2	P3	J7	800
S2	P5	J2	100

P					
PNO	**PNAME**	**PRICE**	**COLOR**	**WEIGHT**	**CITY**
P1	NUT	20.00	RED	12	COLOMBO
P2	BOLT	14.00	GREEN	17	KANDY
P3	SCREW	14.00	BLUE	17	MATARA
P4	SCREW	11.00	RED	14	COLOMBO
P5	CAM	150.00	BLUE	12	KANDY
P6	COG	12.00	RED	19	COLOMBO

S			
SNO	**SNAME**	**STATUS**	**CITY**
S1	SAMAN	20	COLOMBO
S2	JAGATH	10	KANDY
S3	ANIL	30	KANDY
S4	GAMINI	20	COLOMBO
S5	BALA	30	JAFFNA

Get all pairs of supplier numbers such that the two suppliers concerned are located in the same city

```
SELECT FIRST.Sno, SECOND.Sno
FROM S FIRST, S SECOND
WHERE FIRST.CITY =SECOND.CITY
AND FIRST.Sno < SECOND.Sno;
```

character string comparison

S			
SNO	**SNAME**	**STATUS**	**CITY**
S1	SAMAN	20	COLOMBO
S2	JAGATH	10	KANDY
S3	ANIL	30	KANDY
S4	GAMINI	20	COLOMBO
S5	BALA	30	JAFFNA

S			
SNO	**SNAME**	**STATUS**	**CITY**
S1	SAMAN	20	COLOMBO
S2	JAGATH	10	KANDY
S3	ANIL	30	KANDY
S4	GAMINI	20	COLOMBO
S5	BALA	30	JAFFNA

Subqueries/Nested queries

- Some queries require that existing values in a database be fetched and then used in a comparison condition.
- Such queries can be formulated by using nested queries, which are complete {select from where} blocks within the where clause of another query.

Get supplier names for suppliers who supply part P2

```
SELECT SNAME
FROM S
WHERE Sno IN
        (SELECT Sno
        FROM SPJ
        WHERE Pno = 'P2');
```

S			
SNO	**SNAME**	**STATUS**	**CITY**
S1	**SAMAN**	20	COLOMBO
S2	JAGATH	10	KANDY
S3	ANIL	30	KANDY
S4	**GAMINI**	20	COLOMBO
S5	**BALA**	30	JAFFNA

SPJ			
SNO	**PNO**	**JNO**	**QTY**
S1	P1	J1	200
S1	**P2**	J4	700
S2	P3	J2	200
S2	P3	J3	200
S2	P3	J4	500
S2	P3	J5	600
S2	P3	J6	400
S2	P3	J7	800
S2	P5	J2	**100**
S3	P3	J3	200
S3	P4	J4	500
S4	**P2**	J1	300
S4	**P2**	J4	300
S5	P1	J2	1000
S5	**P2**	J2	200
S5	**P2**	J4	100
S5	P3	J4	1200
S5	P4	J4	800
S5	P5	J4	400
S5	P5	J5	500
S5	P5	J7	100
S5	P6	J2	200
S5	P6	J4	500

```
SELECT DISTINCT SNAME
FROM S, SPJ
WHERE S.Sno = SPJ.Sno
AND Pno = 'P2';
```

S			
SNO	**SNAME**	**STATUS**	**CITY**
S1	**SAMAN**	20	COLOMBO
S2	JAGATH	10	KANDY
S3	ANIL	30	KANDY
S4	**GAMINI**	20	COLOMBO
S5	**BALA**	30	JAFFNA

SPJ			
SNO	**PNO**	**JNO**	**QTY**
S1	P1	J1	200
S1	**P2**	J4	700
S2	P3	J2	200
S2	P3	J3	200
S2	P3	J4	500
S2	P3	J5	600
S2	P3	J6	400
S2	P3	J7	800
S2	P5	J2	**100**
S3	P3	J3	200
S3	P4	J4	500
S4	**P2**	J1	300
S4	**P2**	J4	300
S5	P1	J2	1000
S5	**P2**	J2	200
S5	**P2**	J4	100
S5	P3	J4	1200
S5	P4	J4	800
S5	P5	J4	400
S5	P5	J5	500
S5	P5	J7	100
S5	P6	J2	200
S5	P6	J4	500

Get supplier numbers for suppliers who are located in the same city as supplier S1

```
SELECT Sno
FROM S
WHERE CITY =
   (SELECT CITY
         FROM S
          WHERE Sno = 'S1') ;
```

S			
SNO	**SNAME**	**STATUS**	**CITY**
S1	SAMAN	20	COLOMBO
S2	JAGATH	10	KANDY
S3	ANIL	30	KANDY
S4	GAMINI	20	COLOMBO
S5	BALA	30	JAFFNA

Get supplier names for suppliers who are <u>not currently supplying</u> part P2

```
SELECT SNAME
FROM S
    WHERE Sno NOT IN
        ( SELECT Sno
          FROM SPJ
          WHERE Pno = 'P2' );
```

(S1,S4, S5)

SPJ			
SNO	**PNO**	**JNO**	**QTY**
S1	P1	J1	200
S1	**P2**	J4	700
S2	P3	J2	200
S2	P3	J3	200
S2	P3	J4	500
S2	P3	J5	600
S2	P3	J6	400
S2	P3	J7	800
S2	P5	J2	**100**
S3	P3	J3	200
S3	P4	J4	500
S4	**P2**	J1	300
S4	**P2**	J4	300
S5	P1	J2	1000
S5	**P2**	J2	200
S5	**P2**	J4	100
S5	P3	J4	1200
S5	P4	J4	800
S5	P5	J4	400
S5	P5	J5	500
S5	P5	J7	100
S5	P6	J2	200
S5	P6	J4	500

Get the part numbers for <u>all parts in Colombo</u> that are supplied in <u>more than one shipment</u>

```
SELECT Pno
FROM SPJ
WHERE Pno IN
   (SELECT Pno FROM P
    WHERE CITY='COLOMBO')
GROUP BY Pno
HAVING COUNT(*) > 1;
```

Get supplier names for suppliers who supply at least one part where the weight is less than 15.

```
SELECT SNAME
FROM S
WHERE Sno IN
        (SELECT Sno
         FROM SPJ
         WHERE Pno IN
            (SELECT Pno
             FROM P
              WHERE WEIGHT < 15));
```

Get supplier numbers of suppliers whose status value is less than the current maximum status value.

```
Select sno
From S
Where status < (select max(status)
                from S);
```

Retrieval using ANY

Get supplier names for suppliers who supply part P2

```
SELECT SNAME
FROM S
WHERE Sno =ANY
(SELECT Sno
 FROM SPJ
 WHERE Pno = 'P2') ;
```

SNAME
SAMAN
GAMINI
BALA

ATUS	CITY
20	COLOMBO
10	KANDY
30	KANDY
20	COLOMBO
30	JAFFNA

SNO
S1
S1
S2
S2
S2
S2
S2
S2
S2
S3
S3
S4
S4
S5
S5
S5
S5
S5
S5
S5
S5
S5
S5

f = ANY (SELECT F FROM….) evaluates to true if and only if the value <u>f is equal to at least one of the value</u> in the result of evaluating the “SELECT F FROM….” clause.

Similarly the condition

f < ANY (SELECT F FROM….) evaluates to true if and only if the value <u>f is less than at least one value</u> in the result of evaluating the “SELECT F FROM….” clause.

The operators <= ANY, > ANY,>= ANY, != ANY also could be defined.

- Get the supplier numbers for suppliers with status value less than at least the current maximum status value in the S table

```
SELECT Sno
FROM S
WHERE STATUS <ANY
        (SELECT STATUS
         FROM  S) ;
```

SNO
S2
S4
S1

S			
SNO	**SNAME**	**STATUS**	**CITY**
S1	SAMAN	20	COLOMBO
S2	JAGATH	10	KANDY
S3	ANIL	30	KANDY
S4	GAMINI	20	COLOMBO
S5	BALA	30	JAFFNA

```
SELECT Sno
FROM S
WHERE STATUS <
        (SELECT MAX(STATUS)
         FROM  S) ;
```

RETRIVAL USING ALL

- f = ALL (SELECT F FROM….) evaluates to true if and only if the value <u>f is equal to all values</u> in the result of evaluating the “SELECT F FROM….” clause.

```
select SNAME, Status
from S
where STATUS > ALL (select Status
                    from S
                 where city='COLOMBO');
```

S			
SNO	**SNAME**	**STATUS**	**CITY**
S1	SAMAN	20	COLOMBO
S2	JAGATH	10	KANDY
S3	ANIL	30	KANDY
S4	GAMINI	20	COLOMBO
S5	BALA	30	JAFFNA

```
SQL> select *
  2  from branch;
BRANCH_NAME     BRANCH_CITY        ASSETS
--------------- --------------- ----------
Brighton        Brooklyn           7100000
Downtown        Brooklyn           9000000
Mianus          Horseneck          400000
North Town      Rye                3700000
Perryridge      Horseneck          1700000
Pownal          Bennington         300000
Redwood         Palo Alto          2100000
Round Hill      Horseneck          8000000
8 rows selected.
```

```
SQL> select *
  2  from branch
  3  where branch_name > ALL (select branch_name
  4                           from branch
  5                     where branch_city='Brooklyn');
BRANCH_NAME     BRANCH_CITY        ASSETS
--------------- --------------- ----------
Mianus          Horseneck          400000
North Town      Rye                3700000
Perryridge      Horseneck          1700000
Pownal          Bennington         300000
Redwood         Palo Alto          2100000
Round Hill      Horseneck          8000000
6 rows selected.
```

```
SQL> select *
  2  from branch
  3  where branch_name <= ALL (select branch_name
  4                            from branch);
BRANCH_NAME     BRANCH_CITY        ASSETS
--------------- --------------- ----------
Brighton        Brooklyn           7100000
SQL> select *
  2  from branch
  3  where branch_name >= ALL (select branch_name
  4                            from branch);
BRANCH_NAME     BRANCH_CITY        ASSETS
--------------- --------------- ----------
Round Hill      Horseneck          8000000
```

- Similarly the condition

  f < ALL (SELECT F FROM….) evaluates to true if and only if the value <u>f is less than all values</u> in the result of evaluating the “SELECT F FROM….” clause.

  The operators <= ALL, > ALL,>= ALL, != ALL also could be defined.

Get the suppliers with the lowest status value.

```
SELECT SNAME
FROM S
WHERE STAUTS <= ALL (SELECT STATUS
                     FROM S);
```

Get the part number of parts that are currently supplied s.t. the quantity supplied to all projects is greater than 100.

```
Select distinct p1.pno
From SPJ p1
Where 100 < all (select qty
                 from spj p2
                  where p1.pno = p2.pno);
```

PNO
P6
P1
P3
P4

SPJ			
SNO	PNO	JNO	QTY
S1	P1	J1	200
S1	P2	J4	700
S2	P3	J2	200
S2	P3	J3	200
S2	P3	J4	500
S2	P3	J5	600
S2	P3	J6	400
S2	P3	J7	800
S2	P5	J2	100
S3	P3	J3	200
S3	P4	J4	500
S4	P2	J1	300
S4	P2	J4	300
S5	P1	J2	1000
S5	P2	J2	200
S5	P2	J4	100
S5	P3	J4	1200
S5	P4	J4	800

```
Select distinct p1.pno
From SPJ p1
Where 100 < all (select qty
                 from spj p2
                  where p1.pno = p2.pno);
```

PNO
P6
P1
P3
P4

P1				P2			
SNO	PNO	JNO	QTY	SNO	PNO	JNO	QTY
S1	P1	J1	200	S1	P1	J1	200
S1	P2	J4	700	S1	P2	J4	700
S2	P3	J2	200	S2	P3	J2	200
S2	P3	J3	200	S2	P3	J3	200
S2	P3	J4	500	S2	P3	J4	500
S2	P3	J5	600	S2	P3	J5	600
S2	P3	J6	400	S2	P3	J6	400
S2	P3	J7	800	S2	P3	J7	800
S2	P5	J2	100	S2	P5	J2	100
S3	P3	J3	200	S3	P3	J3	200
S3	P4	J4	500	S3	P4	J4	500
S4	P2	J1	300	S4	P2	J1	300
S4	P2	J4	300	S4	P2	J4	300
S5	P1	J2	1000	S5	P1	J2	1000
S5	P2	J2	200	S5	P2	J2	200
S5	P2	J4	100	S5	P2	J4	100
S5	P3	J4	1200	S5	P3	J4	1200
S5	P4	J4	800	S5	P4	J4	800
S5	P5	J4	400	S5	P5	J4	400
S5	P5	J5	500	S5	P5	J5	500
S5	P5	J7	100	S5	P5	J7	100
S5	P6	J2	200	S5	P6	J2	200
S5	P6	J4	500	S5	P6	J4	500

RETRIVAL USING EXISTS

- Whenever a condition in the WHERE clause of a nested query references some attribute of a relation declared in an outer query, the two queries are said to be correlated.
- Nested query is evaluated once for each tuple in the outer query.

RETRIVAL USING EXISTS

- The operator EXISTS is the SQL version of the existential quantifier.

The expression

exists (select * from ……)

Evaluates to true if and only if the result of "select * from ….." <u>is not empty</u>

RETRIVAL USING EXISTS

Get supplier names for suppliers who supply part P2.

```
SELECT SNAME
FROM S
WHERE EXISTS
      (SELECT *
       FROM SPJ
       WHERE SPJ.Sno = S.Sno AND SPJ.Pno = 'P2');
```

SNAME
SAMAN
GAMINI
BALA

EXISTS - represents the existential quantifier.

```
SELECT DISTINCT(SNAME)
FROM S, SPJ
WHERE SPJ.Sno = S.Sno AND SPJ.Pno = 'P2';
```

RETRIVAL USING <u>NOT EXISTS</u>

- The operator NOT EXISTS is the complement form of EXISTS

The expression

not exists (select * from ……)

Evaluates to true if and only if the result of "select * from ….." <u>is empty</u>

Get supplier names for suppliers who are not currently supplying part P2

```
SELECT SNAME
FROM S
    WHERE NOT EXISTS
        ( SELECT *
          FROM SPJ
          WHERE Sno = S.Sno AND Pno = 'P2' );
```

Get the supplier names for suppliers who currently supply all parts

A supplier supplies all parts if

- there is no part
- that the supplier does not supply!

```
SELECT SNAME
FROM S
WHERE NOT EXISTS
      (SELECT *
        FROM P
        WHERE NOT EXISTS
         (SELECT *
          FROM SPJ
          WHERE SPJ.Sno  = S.Sno AND
                         SPJ.Pno = P.Pno);
```

```
SELECT SNAME
FROM S
WHERE NOT EXISTS
    ( SELECT *
      FROM P
      WHERE  (S.Sno, P.Pno) NOT IN
        (SELECT Sno, Pno FROM SPJ)
    );
```

Get the supplier name for suppliers who currently supply all parts that are supplied by S1

```
SELECT SNAME
FROM S
WHERE NOT EXISTS
    ( SELECT *
      FROM P
      WHERE  Pno  IN
          (SELECT Pno FROM SPJ
                WHERE Sno = 'S1')
       AND  NOT EXISTS
            (SELECT *
             FROM SPJ
             WHERE SPJ.Sno = SPJ.S.Sno AND Pno = P.Pno));
```

Get the supplier name for suppliers who currently supply all parts that are supplied by S1

Get the supplier name for suppliers for whom there is some part that is not being supplied by the supplier

```
SELECT SNAME
FROM S
WHERE EXISTS
    ( SELECT *
      FROM P
      WHERE  NOT EXISTS (SELECT *
                         FROM SPJ
                          WHERE SPJ.SNO = S.SNO
                           AND SPJ.PNO = P.PNO));
```

```
SELECT SNAME
FROM S
WHERE EXISTS
    ( SELECT *
      FROM P
      WHERE  (S.Sno, P.Pno) NOT IN
          (SELECT Sno, Pno FROM SPJ);
     );
```

RETRIVAL USING UNION

Get part numbers for parts that either weigh more than 18 pounds or are currently supplied by supplier S2.

```
SELECT   Pno       FROM P
WHERE  WEIGHT > 18

UNION

SELECT    Pno      FROM SPJ
WHERE Sno = 'S2';
```

Top predicate

Select the top seven shipments.

```
Select top 7 *
From spj
Order by qty desc;
```

- Parameters

You can specify parameters to be resolved at run time.

Get the details of all shipments s.t. the quantity is greater than a given value.

```
parameters g_value]number;

select *
from spj
where qty > g_value;
```

Select into statement

Creates a new table and inserts values.

```
select s_no, city  into cs
from S
where city = 'colombo';
```

```
Create table cs
as select sno, city
from S
where city = ‘COLOMBO';
```

Note: the primary key is not defined.

Insert into statement

Use to insert a singe records or multiple records

```
Insert into cs
Select sno, city
From S
Where city = 'KANDY';
```

Note: CS is an existing table.

Drop statement

Removes an existing table from the database

Drop table S;

Alter statement

- Add column
- Delete a column
- Add/delete constrains

```
ALTER TABLE CS
ADD SupName VARCHAR(10);

alter table CS
drop column SupName;
```

Update Operations

Change the price of part P2 to 15 and increase its weight by 5

```
UPDATE P
SET PRICE = 15,
   WEIGHT = WEIGHT +  5
WHERE Pno = ‘P2’;
```

Double the status of all suppliers in Colombo

```
UPDATE S
SET STATUS = 2 * STATUS
WHERE CITY = ‘COLOMBO';
```

Set the quantity to zero for all suppliers in Colombo.

```
UPDATE SPJ
SET QTY = 0
WHERE 'COLOMBO' =
        (SELECT CITY
         FROM S
         WHERE Sno = SPJ.Sno);
```

Deletion

Delete supplier S1

```
DELETE
FROM S
WHERE Sno = 'S1';
```

Notice here that we have a problem with referential integrity

Delete all shipments

```
DELETE
FROM SPJ;
```

Note that with this query there is no problem with referential integrity, since SPJ is not the *referenced* table

- Delete all shipments from suppliers in COLOMBO.

```
DELETE
FROM SPJ
WHERE 'COLOMBO' =
            (SELECT CITY
              FROM S
             WHERE Sno = SPJ.Sno);
```

Creating domain in SQL

- When tables are created it is possible to specify the data type of each attribute directly as given in the CREATE TABLE Command. (see slides 8 and 9 of SQL part1).
- Alternatively, a domain can be declared and the domain name used with the attribute specification.

CREATE DOMAIN sup_type AS CHAR(2);

We can use sup_type for the attribute Sno of table S and table SPJ.

A domain also can have an optional default specification via a DEFAULT clause.

Constraints in SQL

- Here we look at the basic constraints that can be specified in SQL as part of table creation. These include
  - Key and referential integrity constraint
  - Restrictions on attribute domains
  - Constraints on individual tuples within a relation.

Attribute constraints and Attribute defaults.

- A constraint NOT NULL can be specified if NULL is not permitted for a particular attribute.
- Can define a default value for an attribute by appending a clause DEFAULT <value>
- Can restrict the attribute or domain values using the CHECK clause following an attribute or domain definition.

```
STATUS NUMBER NOT NULL CHECK
   (STATUS > 0 AND STATUS < 50);

CREATE DOMAIN sup_status AS NUMBER
CHECK (sup_status > 0 AND
                          sup_status < 50);
```

Key and Referential Integrity Constraint

```
CREATE TABLE SPJ
 ( Sno  Char(2) NOT NULL,
   Pno  Char(2) NOT NULL   DEFAULT 'P1',
   Jno   Char(2) NOT NULL);

alter table S add constraint sconsPK Primary key (SNO);
alter table P add constraint pconsPK Primary key (PNO);
alter table J add constraint jconsPK Primary key (JNO);
alter table SPJ add constraint
SPJ_PK primary key (SNO, PNO, JNO);

Alter table SPJ add CONSTRAINT fk_con1 FOREIGN KEY (Sno)
    REFERENCES  S  (Sno)
    ON UPDATE NO ACTION
    ON DELETE CASCADE;
```

```
Alter table SPJ add CONSTRAINT fk_con2
  FOREIGN KEY (Pno)
    REFERENCES P (Pno)
    ON UPDATE CASCADE
    ON DELETE SET DEFAULT ;

Alter table SPJ add CONSTRAINT fk_con3
  FOREIGN KEY (Jno)
    REFERENCES  J  (Jno)
    ON UPDATE CASCADE
    ON DELETE CASCADE;
alter table SPJ drop constraint SPJ_PK;
```

```
Create Table SPJ
(sno char(2) not null,
 pno char(2) not null,
 jno char(2) not null,
 primary key (sno, pno, jno),
 constraint fkSPJ1 foreign key (sno) references S (SNO),
 constraint fkSPJ2 foreign key (pno) references P (PNO),
 constraint fkSPJ3 foreign key (jno) references J (JNO)
 );
```

- The options for ON UPDATE and ON DELETE are

  CASCADE,

  SET NULL,

  SET DEFAULT, and

  NO ACTION

- These are the compensating actions that the system will perform in order to (try to) maintain referential integrity

- NO ACTION is the default.

- If a deletion/update would violate referential integrity, then NO ACTION will prevent the deletion/update
  - since no (compensating) action is performed

- Exercise :

Write the create table command for each relation in the Company Database.

```
SQL> insert into bc
  2  select *
  3  from branch
  4  where branch_city = 'Brooklyn';

2 rows created.


SQL>  select *
  2   from bc
  3     where branch_name =ALL
(select branch_name
  4             from branch
  5    where
branch_city='Brooklyn');
no rows selected
```

```
SQL> select branch_name
  2            from branch
  3   where branch_city='Brooklyn';
BRANCH_NAME
---------------
Brighton
Downtown

SQL> select *
  2  from bc;
BRANCH_NAME     BRANCH_CITY         ASSETS
--------------- --------------- ----------
Brighton        Brooklyn            7100000
Downtown        Brooklyn            9000000

SQL>  select *
  2   from bc
  3     where branch_name <ALL (select branch_name
  4              from branch
  5     where branch_city='Brooklyn');
no rows selected
```

```
SQL> select *
  2  from bc
  3    where branch_name <=ALL (select branch_name
  4              from branch
  5     where branch_city='Brooklyn');
```

```
BRANCH_NAME     BRANCH_CITY        ASSETS
--------------- --------------- ----------
Brighton        Brooklyn          7100000
```

```
SQL> select *
  2  from bc
  3    where branch_name >=ALL (select branch_name
  4              from branch
  5      where branch_city='Brooklyn');
```

```
BRANCH_NAME     BRANCH_CITY        ASSETS
--------------- --------------- ----------
Downtown        Brooklyn          9000000
```

```
SQL>  select *
  2   from bc
  3     where branch_name !=ALL (select branch_name
  4               from branch
  5       where branch_city='Brooklyn');

no rows selected
```